

# 演算法視覺化手冊

圖解 + 用途說明 + 好處 / 壞處 · Visual Algorithm Handbook

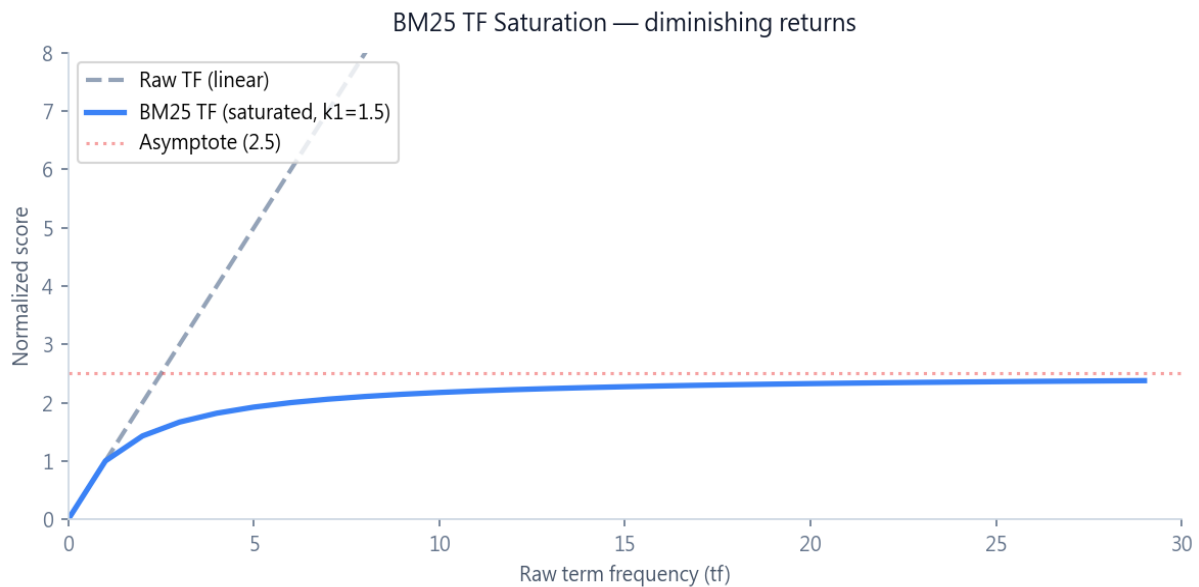
串連系統 v2.2 · 投資公司管理層決策輔助

本手冊以「圖 + 說明」的方式呈現系統 15 個核心演算法。每個演算法包含：

- (1) 示意圖 — 視覺化說明演算法本質
- (2) 為什麼這樣用 — 設計動機與學理依據
- (3) 好處 ✓ — 為什麼選這個演算法
- (4) 壞處 / 限制 — 缺點與權衡

適合期末口試準備、答辯素材、論文方法章節。

## #1 BM25F — Field-weighted BM25 · 資訊檢索



### 為什麼這樣用

傳統 TF-IDF 採線性 TF 計分，「出現 5 次」與「50 次」差距過大；BM25 用飽和函數  $tf \times (k1+1) / (tf + k1 \times len\_norm)$  讓 TF 在多次出現後趨於飽和，更接近人類認知。F (Field) 表示對不同欄位給予不同權重 — 標題出現「東京中央銀行」遠比內文出現重要。

### 好處 ✓

- TF 飽和符合直覺：同詞出現 5 次和 50 次的相關性差距不該是 10 倍
- 欄位權重讓「短而精準的標題命中」勝過「長文裡偶然出現」
- Lucene / Elasticsearch 同款，業界已驗證的成熟演算法
- 純前端跑、零外部 API、零成本、保護機密

### 壞處 / 限制

- 需要事先設定欄位權重表（標題 5.0、tags 4.0、...）— 是工程經驗值
- 對極短文件（如只有標題）較不利，因長度正規化會把分數拉低
- 不理解語意：「投資」與「投入資金」靠詞面相似度，無法理解抽象同義
- 資料量 >10k 筆會慢，那時要遷移到真正的 Elasticsearch

## #2 Multi n-gram Chinese Tokenization · 資訊檢索

## Multi n-gram Tokenization (Chinese has no word boundary)

### 東京中央銀行

1-gram 東 京 中 央 銀 行

2-gram 東京 京中 中央 央銀 銀行

3-gram 東京中 京中央 中央銀 央銀行

3-gram catches proper nouns like 'investment-committee'

#### 為什麼這樣用

中文沒有空格詞界 (vs 英文 "Tokyo Central Bank" 自然以空格分詞)，Jieba 等斷詞器又對新詞 / 專有名詞容易斷錯。本系統同時切 1-gram / 2-gram / 3-gram，讓任何字串片段都能被精準召回。

#### 好處 ✓

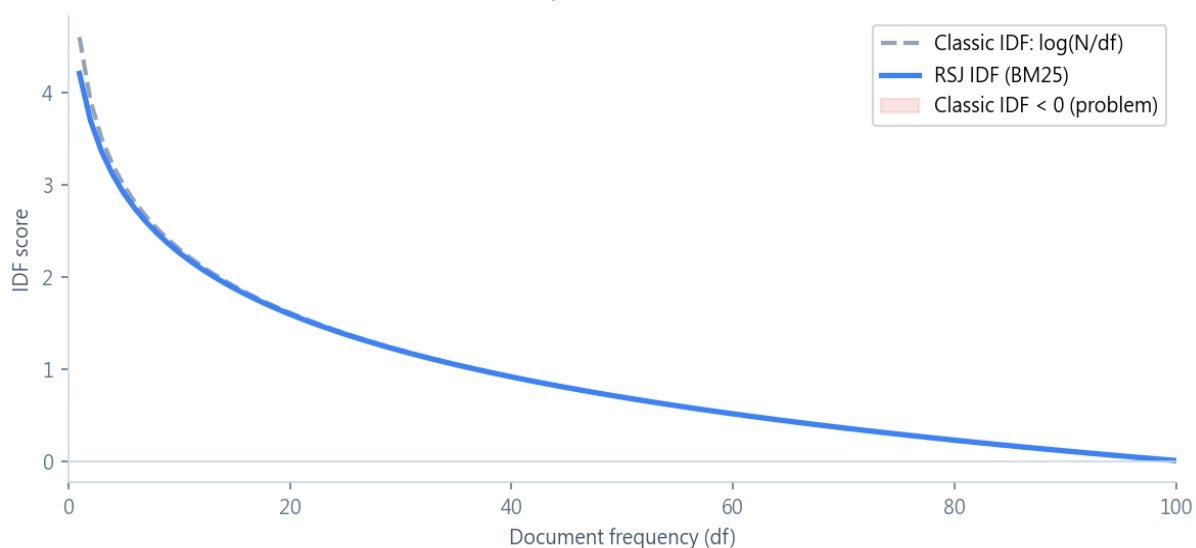
- 對新詞 / 專有名詞容忍度高 — 不依賴詞典即可匹配
- 3-gram 可捕捉 3-4 字術語 (投委會、董事會、伊勢島飯店)
- 簡單可靠、無外部相依、計算成本低

#### 壞處 / 限制

- Token 數量爆增：6 字詞會切出 15 個 token，索引變大
- 會產生無意義的 token (如「京中央」) — 但 IDF 會把它們的權重降到很低，影響不大
- 對英文沒有 stemming (investing 與 investment 不互通) — 需另外加 Porter Stemmer

## #3 Robertson-Sparck-Jones IDF · 資訊檢索

Robertson-Sparck-Jones IDF vs Classic



#### 為什麼這樣用

經典  $IDF = \log(N/df)$  在  $df > N/2$  時會變負數（極常見詞 stop word 被罰太重），BM25 改用 RSJ 公式  $\log(1 + (N - df + 0.5)/(df + 0.5))$ ，加 Lidstone smoothing 確保結果穩定且永遠  $> 0$ 。

好處 ✓

- 永遠非負，可放心乘上 TF 而不會反向扣分
- 加 +0.5 smoothing 避免邊界除零 / 極端值問題
- 對 stop word 自然趨近 0 分（達成「自動忽略 stop word」效果）

壞處 / 限制

- 公式不如經典  $\log(N/df)$  直觀，初學者要先學會
- +0.5 是經驗常數，不同領域可能需要微調

## #4 Synonym Normalization · 資訊檢索

### Synonym Normalization (14 groups)

|                                |   |         |
|--------------------------------|---|---------|
| 募資 / 融資 / 募款 / fundraising     | → | → 募資    |
| 盡調 / 盡職調查 / DD / due diligence | → | → 盡調    |
| NDA / 保密協議 / 保密                | → | → NDA   |
| 投委會 / 投資委員會 / IC               | → | → 投委會   |
| 退場 / exit / 出場                 | → | → 退場    |
| Pre-A / PreA / 種子輪後            | → | → Pre-A |

Replace longest first to avoid substring overwrite (e.g., '盡職調查' before '盡調')

為什麼這樣用

金融 / 投資領域同個概念有多種寫法（盡調 vs DD vs Due Diligence），若不處理，「DD」搜不到含「盡職調查」的案件。Tokenize 前統一轉成 canonical form 解決此問題。

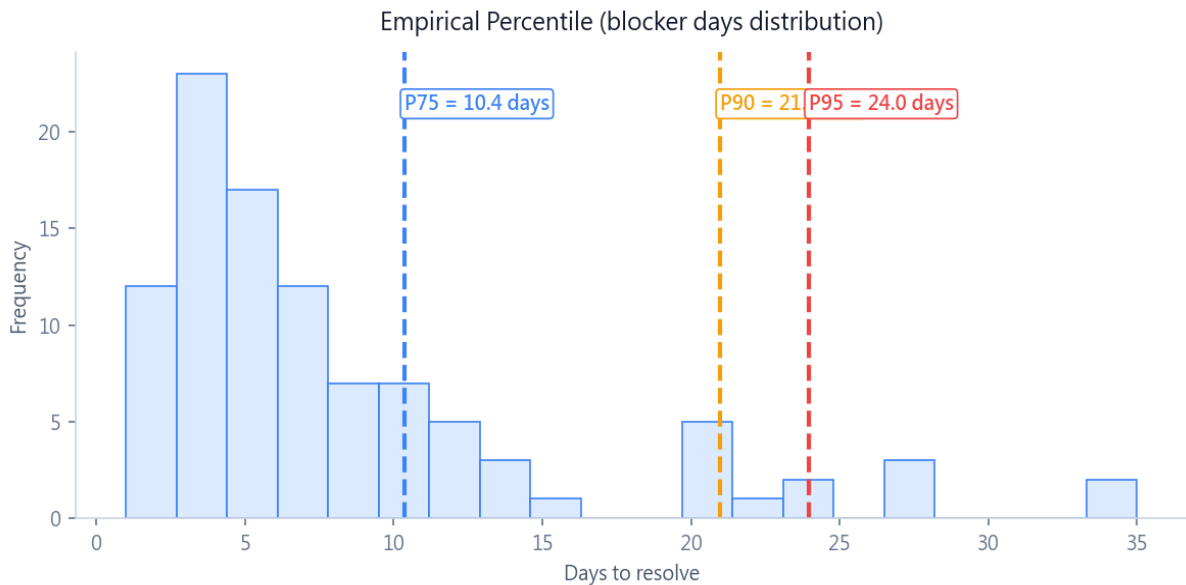
好處 ✓

- 立即提升召回率（recall）— 不用使用者背所有縮寫
- 完全可控、可解釋（明確列出每組對應）
- 從長到短替換確保不會誤切（先替「盡職調查」再替「盡調」）

壞處 / 限制

- 需要人工維護同義詞表，新詞要手動加
- 可能造成過度匹配（「客戶」與「customer」可能被當完全同義，但有時 customer 指外部）
- 不能處理多義詞（「銀行」可能指金融機構或河岸）

## #5 Empirical Percentile · 統計分析



### 為什麼這樣用

不同類別卡點的合理時長差異極大（法遵 7-8 天合理、跨部門 4-5 天就該升級）。用「同類歷史」的分位數判風險，比「絕對 10 天」這種固定門檻更精準、更自適應。

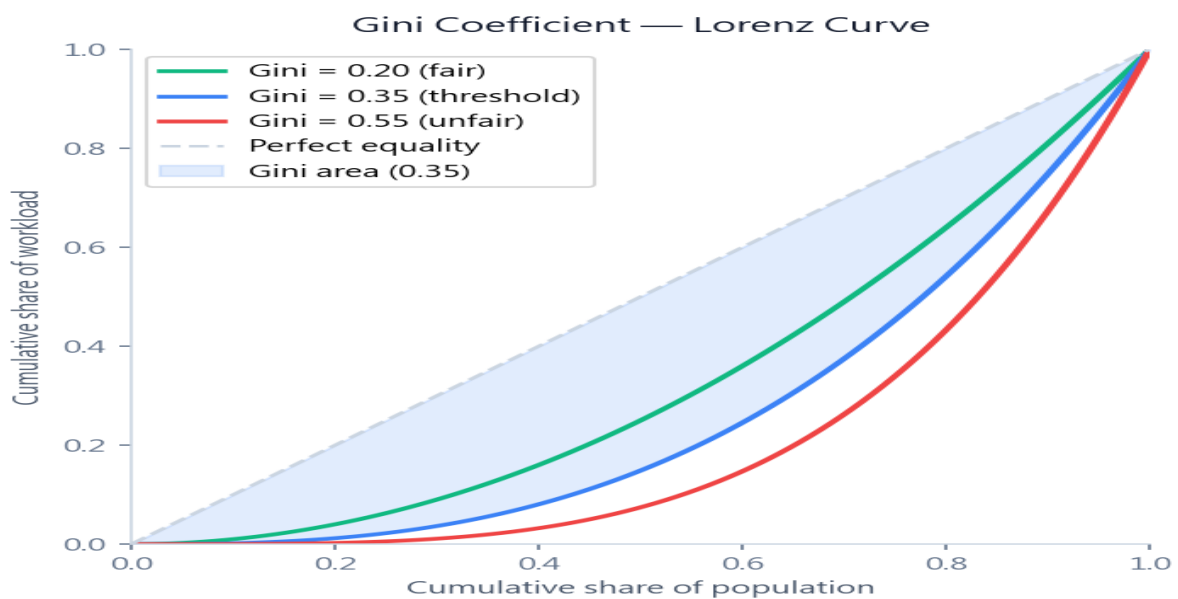
### 好處 ✓

- 自適應公司規模 / 產業淡旺季 — 不需重新調參
- 對離群值 (outlier) 魯棒 — 不像 mean+std 易被極端值拉偏
- 線性內插提供連續數值，避免階梯式跳變

### 壞處 / 限制

- 樣本太少 (< 5 筆) 會不穩定 — 需 fallback 到全公司歷史
- P95+ 在歷史長尾極稀疏 — 邊界估計誤差大
- 對極新類別（從沒解過的卡點種類）完全無歷史可比

## #6 Gini Coefficient · 統計分析



### 為什麼這樣用

經濟學量化「分配不均」的標準工具。本系統用來衡量員工負載不均度 — 若  $Gini > 0.35$  代表工作量集中在少數「超人」身上，組織單點失敗風險高（Key Man Risk）。

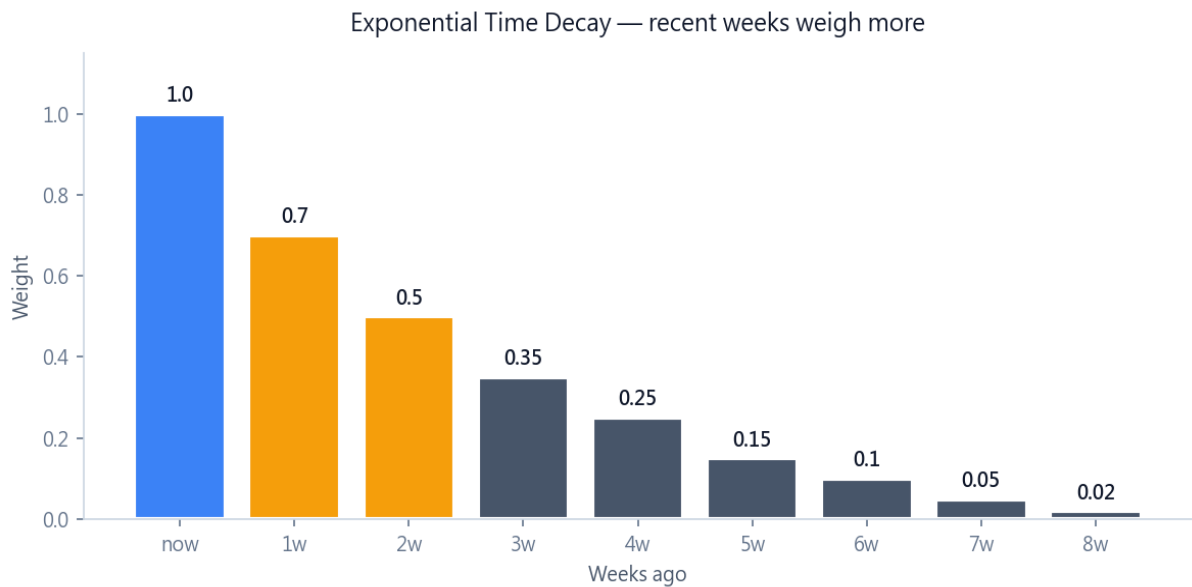
#### 好處 ✓

- 0.35 是經濟學公平 / 不公平的學術分界 — 有理論依據
- 對總量不敏感 — 公司大小不同都能比較
- 排序後  $O(n)$  公式比標準  $O(n^2)$  公式快很多

#### 壞處 / 限制

- 純不均度指標，看不出「是高負載者太高還是低負載者太低」
- 對小團隊（< 5 人）容易跳動 — 一人變化就大幅影響
- 假設「平均負載 = 健康」，但某些角色本來就該高負載（如 CTO）

## #7 Exponential Time Decay · 時間序列



### 為什麼這樣用

員工負載分析中，本週的案件壓力遠大於8週前的案件。用 [1.0, 0.7, 0.5, 0.35, ...] 指數衰減陣列做時間加權，符合「近期工作影響更大」的人類直覺。

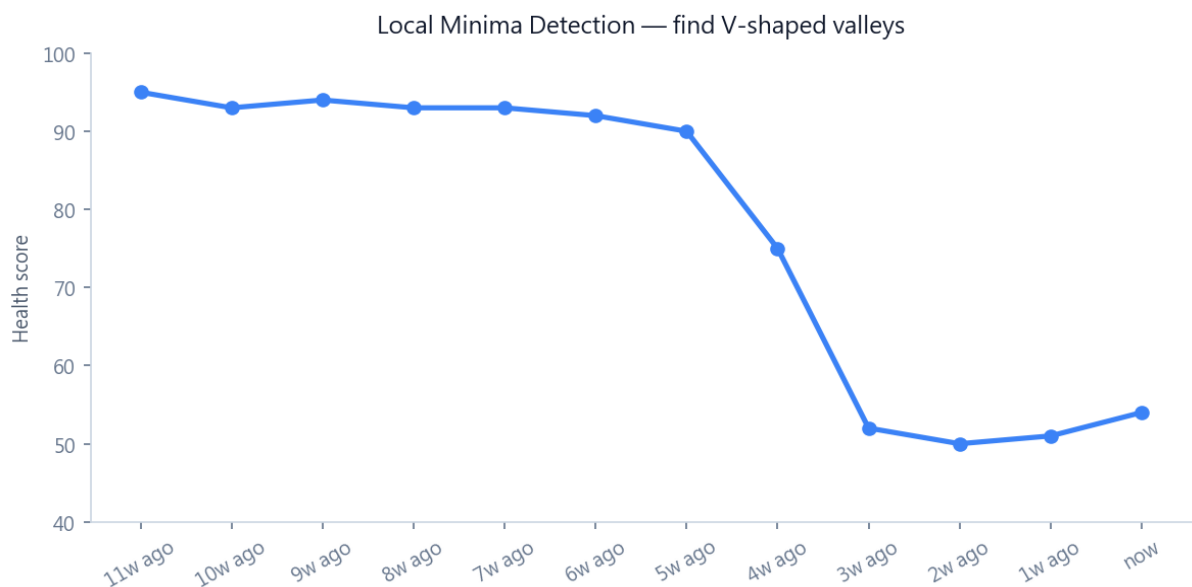
### 好處 ✓

- 近期權重高、遠期權重低，貼近實際感受
- 9週前後一律 0，自然限定計算範圍
- 離散查表  $O(1)$  計算，零開銷

### 壞處 / 限制

- 權重是寫死的經驗值，理想應該依「案件類別」動態調整
- 8週前 → 9週前突然從 0.02 跳到 0 — 階梯邊緣
- 對「跨年大案」這種一直拖很久的不公平 — 會被低估

## #8 Local Minima Detection (拐點偵測) · 時間序列



## 為什麼這樣用

管理層需要快速找出組織健康度何時開始崩跌 — 「拐點」就是 V 型谷底。雙邊閾值 -3 分要求左右兩側都明顯較高，過濾隨機雜訊。

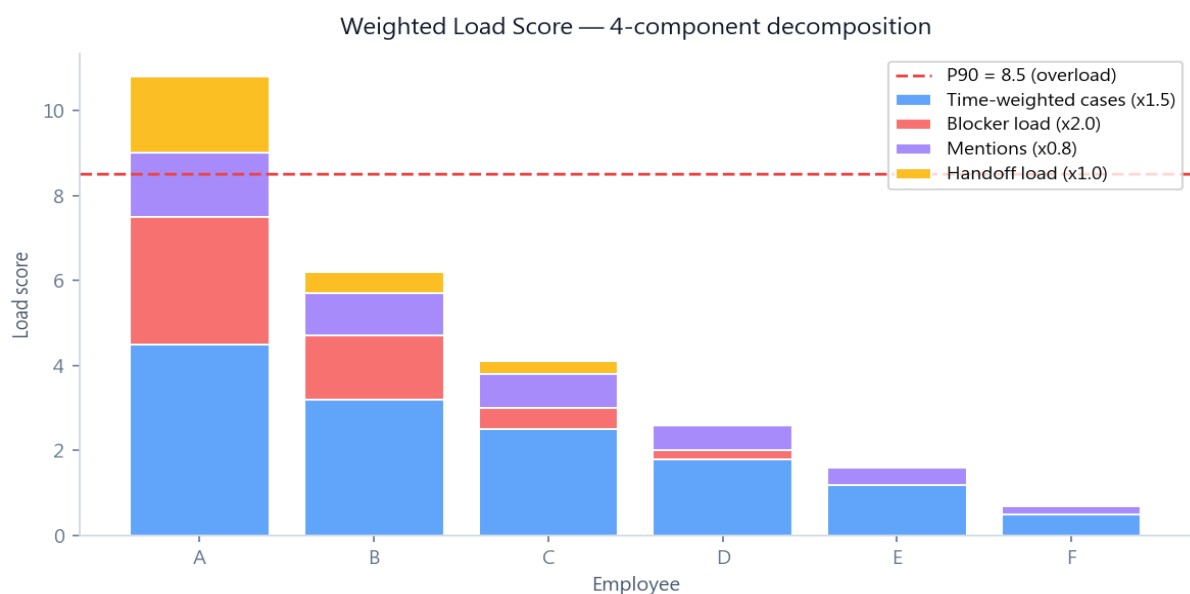
### 好處 ✓

- 簡單可解釋：每個點看左右兩鄰居即可
- 雙邊對稱閾值排除單邊偶然下跌（一次性雜訊）
- $O(n)$  一次掃描完成

### 壞處 / 限制

- 閾值 3 分是 magic number — 在分數區間  $[0,100]$  經驗值
- 找不到「緩慢長期下滑」這種無明顯谷底的情境
- 不知道為什麼掉 — 只標位置，原因要去看當週事件

## #9 Weighted Load Score · 加權評分



## 為什麼這樣用

員工的負載不只是「處理幾件案件」 — 還有卡點數、被提及次數、交接負擔。用 4 元素加權融合：卡點  $\times 2.0$ （最重，代表正在燒）、案件  $\times 1.5$ 、交接  $\times 1.0$ 、提及  $\times 0.8$ 。

### 好處 ✓

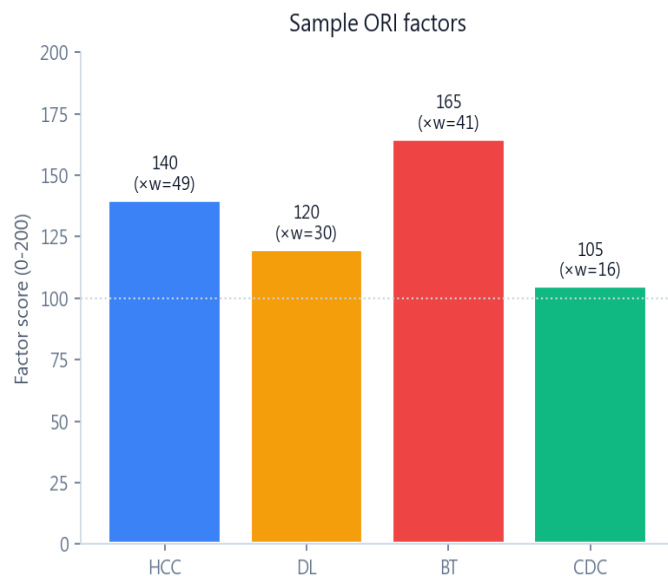
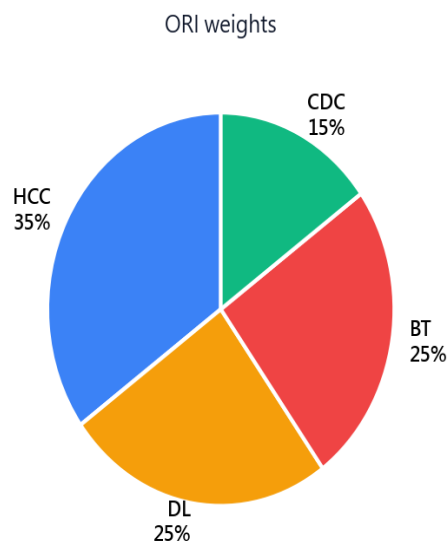
- 多維度反映真實工作壓力，不只看表面案件數
- 卡點權重最高 — 因為卡點代表「需要立刻關注」
- Percentile rank 判定過載  $\rightarrow$  自適應公司規模

### 壞處 / 限制

- 4 個權重 (1.5/2.0/0.8/1.0) 是經驗值，沒有完美公式
- 從週報文字偵測「卡」「延」等關鍵字 — 對白話文或英文夾雜不穩
- 新員工沒歷史資料  $\rightarrow$  計算困難

## #10 ORI — Organizational Risk Index · 加權評分





### 為什麼這樣用

給管理層的「組織體溫計」— 融合人力集中度 (HCC)、決策延遲 (DL)、卡點長尾風險 (BT)、跨部門協作 (CDC) 四大因子。HCC 權重最高 (35%) 因為人力集中是組織單點失敗主因。

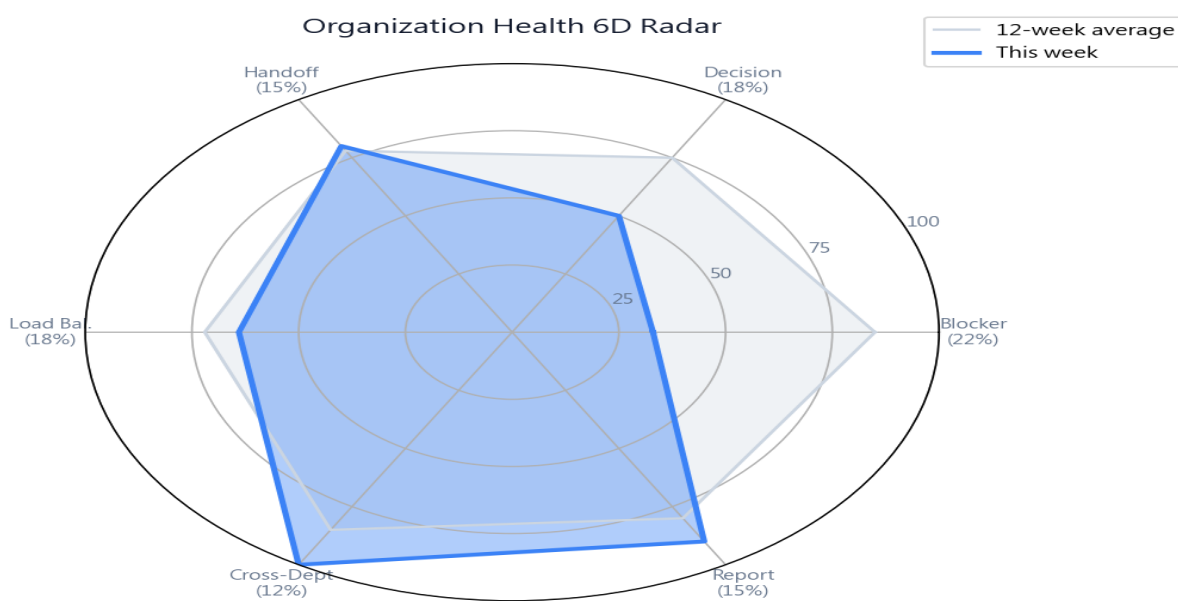
### 好處 ✓

- 0-200 反向計分 — 越高越警示，符合 risk index 慣例
- 四因子涵蓋組織健康主要面向
- 五級告警對應具體建議文案，可直接行動

### 壞處 / 限制

- 0-200 對直覺管理層不友善（多數人習慣 0-100）— 故 v2.1 補了 Org Health 6D
- 權重 (35/25/25/15) 是經驗值，理應依公司階段調整
- 因子之間可能有相關性（例如過載通常伴隨卡點），未做共線性分析

## #11 Organization Health 6D Radar · 加權評分



### 為什麼這樣用

管理層友善的 0-100 正向計分版本。6 維雷達涵蓋卡點、決策、交接、負載、協作、週報。搭配 12 週趨勢線 + 拐點偵測 + 點擊事件 inline 展開 — Plan→Decide→Track→Learn 閉環的核心儀表板。

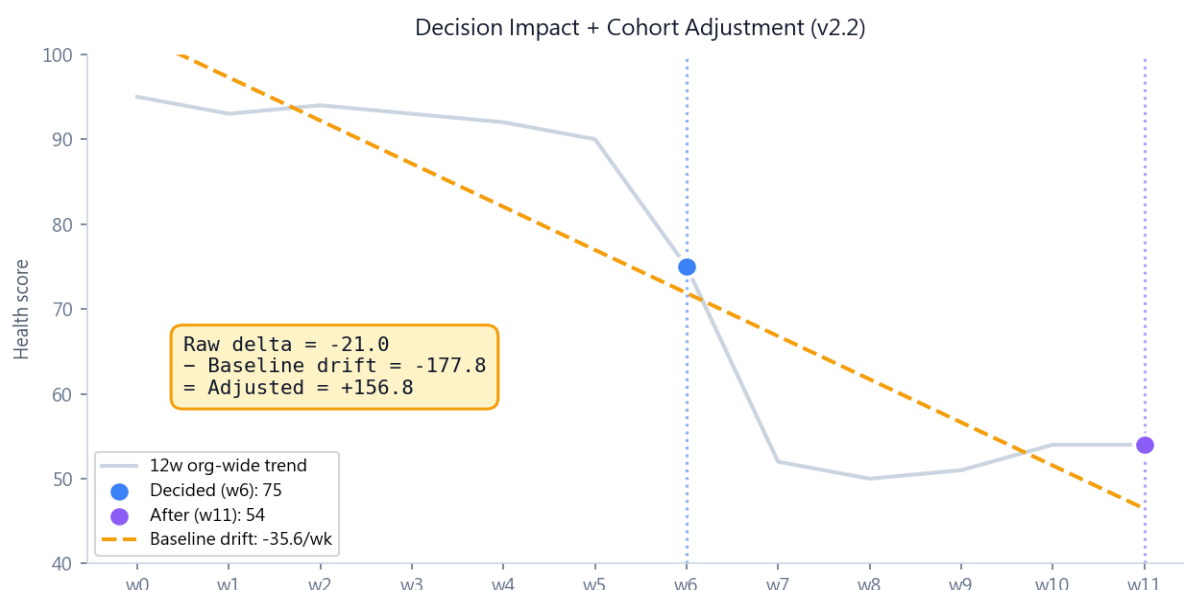
好處 ✓

- 雷達圖直觀展示「強項 / 弱項」，一眼看出該補哪個維度
- 本週 vs 12 週均值雙圖層比對 — 看出短期偏差
- 底層共享同一組分析器，跨頁面數字保證一致

壞處 / 限制

- 權重 (0.22/0.18/0.15/0.18/0.12/0.15) 是專家會議值，不同產業適配性需驗證
- 週報品質維度容易被刷分（多寫廢話也能拉長度）
- 若公司未啟動週報制度，週報品質永遠 0 分 — 拉低整體

## #12 Decision Impact + Cohort Adjustment (v2.2) · 加權評分 ★



為什麼這樣用

原本只看「決策完成後 N 週的健康度變化」，會被大環境趨勢汙染（整體下滑期決策都被冤枉）。v2.2 引入 Cohort Adjustment：扣掉「同期基準漂移」，得出純粹歸因於該決策的影響。

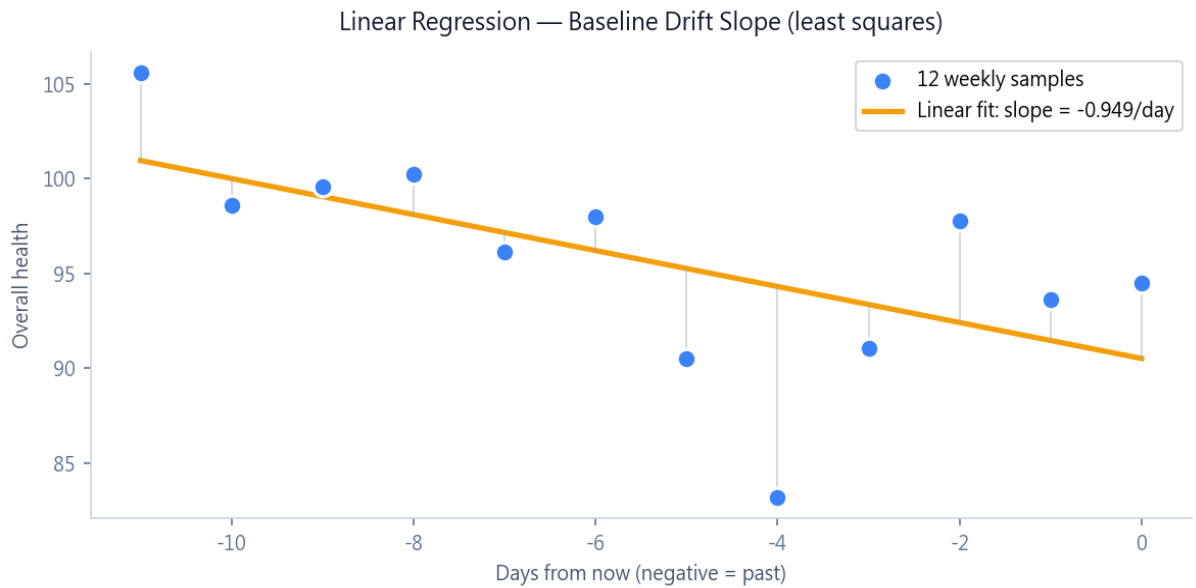
好處 ✓

- 解決因果歸因問題 — 區分「決策自身效果」vs「大環境趨勢」
- 用 12 週線性回歸算 baseline drift，O(1) 攤平多筆決策計算
- 正面 / 中性 / 負面三級判定，配合排行版直觀化

壞處 / 限制

- Baseline drift 假設組織趨勢是線性 — 實際可能有突變
- 完成不到 4 週 (windowWeeks) 的決策標記「追蹤中」— 給暫評
- 對「靜默成功」（防止惡化但沒推升）的決策可能低估

## #13 Linear Regression Baseline Drift Slope (v2.2 工具) · 時間序列



### 為什麼這樣用

Cohort Adjustment 的核心工具。從 12 週快照採樣 12 個 (x, y) 點，用最小平方法 (Least Squares) 算出大盤每日漂移率。比「首尾相減 / 天數」對單週雜訊魯棒得多。

### 好處 ✓

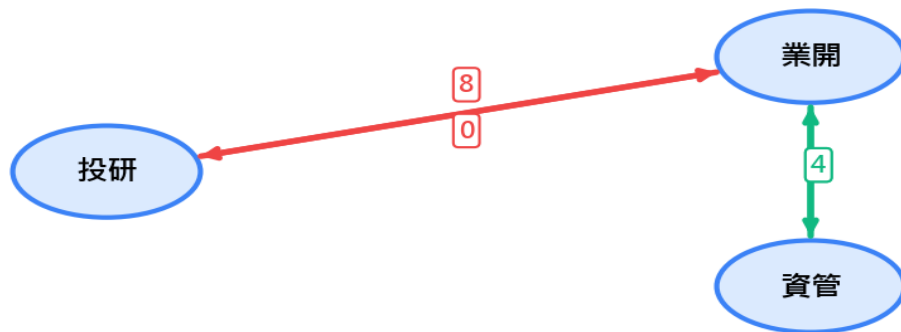
- 12 點互相抵消雜訊，找出穩健的長期趨勢
- 純數學，無 hyperparameter — 確定性結果
- 計算  $O(n)$  一次性，driftCache 共享給所有決策避免重算

### 壞處 / 限制

- 假設線性趨勢 — 若組織有 S 型成長或週期性，會失真
- 12 週樣本對「跨季度」事件可能不夠
- Slope 對首尾極端點敏感 (leverage effect)

## #14 Asymmetric Communication Detection · 圖論

### Asymmetric Communication Detection



★ Red pair: 投研 calls 業開 8 times, but 業開 never calls 投研 → ASYMMETRIC (organizational issue)  
□ Green pair: 業開 & 資管 communicate both ways → HEALTHY

#### 為什麼這樣用

識別組織內的「溝通黑洞」— A 一直找 B ( $\geq 5$  次) 但 B 完全沒回應

A。這是組織壁壘、推託、流程斷層的明確徵兆，計入 ORI 的 CDC 與健康度的部門協作維度。

#### 好處 ✓

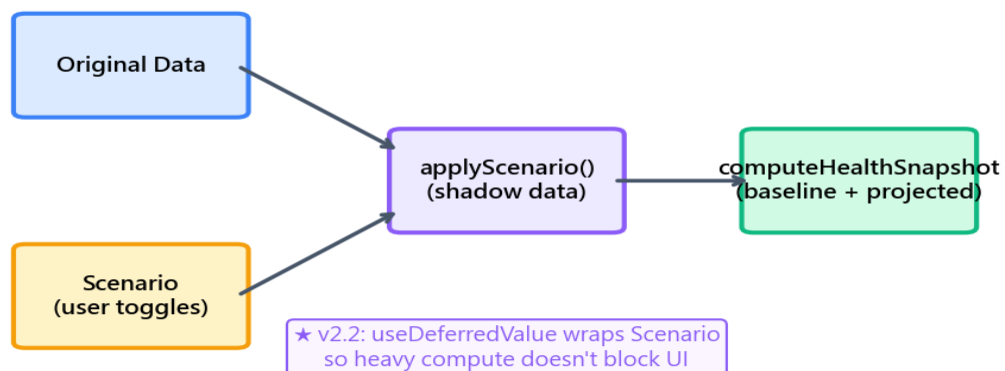
- 明確的組織病徵偵測 — 不只看流量，更看「對稱性」
- 閾值 5 過濾偶然提及，只標真正單向依賴
- 圖論基礎，可擴展到更複雜的網絡指標（中心性、社群偵測）

#### 壞處 / 限制

- 閾值 5 是 magic number，小團隊可能不適用
- 未區分「該回但沒回」vs「本來就不該回」的對話
- 依賴週報文字 mention 抽取，若某部門寫週報少會偵測不到

## #15 What-if Scenario Simulation (v2.2) · 預測模擬

### What-if Simulation Flow



## 為什麼這樣用

管理層在做決策前需要「先看後果」。What-if 提供互動式 sandbox — 拉滑桿模擬「解掉這 3 個卡點 + 加 2 名法遵專員」會讓組織健康度變多少。v2.2 加 useDeferredValue 避免重算卡 UI。

## 好處 ✓

- Demo 神器 — 教授拉開關就感受演算法價值
- 對原始資料 fork shadow data, 不汙染主資料
- React 19 useDeferredValue 自動降優先級, UI 流暢
- 與 baseline 雙圖層雷達比對 + delta 智能建議文案

## 壞處 / 限制

- 假設「解掉卡點 = 立即生效」, 現實中可能有延遲
- 模擬「加員工」目前是新增 loadScore=0 的人, 可能拉高 Gini (反效果) — 該改為 reassignment
- 資料量 10x 時即使 useDeferredValue 也會卡, 需 Web Worker

## 結語：演算法選擇的設計哲學

本系統 15 個核心演算法（加上 13 個工具機制，共 28+1）共同實現一個目標：  
「用恰當的演算法解決恰當規模的問題」。

### 為什麼不用 LLM / Embedding？

- (1) 資料量小：53 筆歷史案、240 筆週報，BM25F 比 Embedding 更可靠且零成本。
- (2) 可解釋性：管理層需要知道「為什麼推薦這筆」，BM25F 可逐項列出命中詞貢獻。
- (3) 機密敏感：投資公司資料不能送 cloud API。
- (4) 計算成本：BM25F、Gini、percentile 都是  $O(N)$  或  $O(N \log N)$ ，瀏覽器毫秒級。
- (5) 確定性：相同輸入永遠回傳相同結果，LLM 有 stochasticity。

### 好處 / 壞處的核心權衡

#### 用「經驗值權重」而非「機器學習」

整套系統大量使用人工調參的權重（field weights、time decay、ORI weights 等）。好處：可解釋、可調、無黑盒；壞處：需要領域專家定參數，新公司導入要 1-2 週調參。這個選擇對「中小型公司、決策層使用」最適合。

#### 用「Empirical Percentile」而非「絕對門檻」

好處：自適應公司規模、自適應產業；壞處：樣本不足時不穩定。對於 50 人規模、200 筆歷史的中型企業，這是最佳折衷。

#### 用「動態判定 helper」而非「status 字串」

v2.1 把所有「逾期」判定改成動態日期計算，不再依賴 status 字串。好處：自動偵測「dueDate 過期但 status 沒更新」；壞處：每次都要重算，輕微 CPU 開銷。對使用者體驗有正面影響（不會看到狀態錯亂）。

「資料越少，演算法的選擇越重要。」

「跨頁面的一致性，比單頁的炫技更重要。」

「能解釋的演算法，比準確 1% 的黑盒更值錢。」

—— 串連系統 v2.2 設計哲學